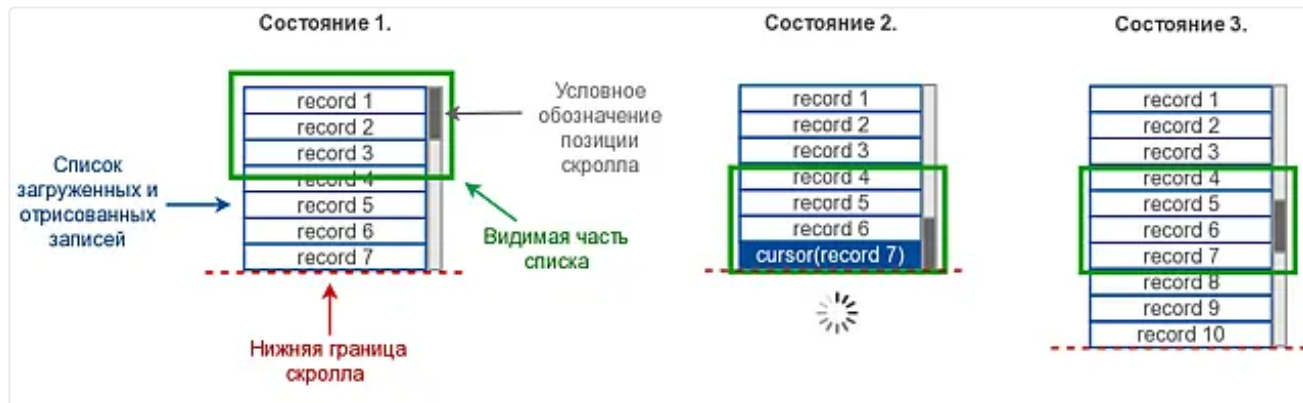


Навигация по курсору

Навигация по курсору является механизмом ускоренной выборки данных из таблицы БД относительно т.н. *курсора* (фиксированной записи). В отличие от постраничной навигации, для выборки не требуется обработка ранее загруженных данных, что в результате снижает нагрузку на БД.



На рисунке показан пример скролла вниз. В состоянии № 2 видимая часть списка достигает нижней границы скролла, а последняя запись — record 7 — становится курсором. Относительно неё производится выборка и отрисовка новых записей вниз, что показано в состоянии № 3. При последующем скролле вниз курсором станет record 10.

Конфигурация бизнес-логики приложения

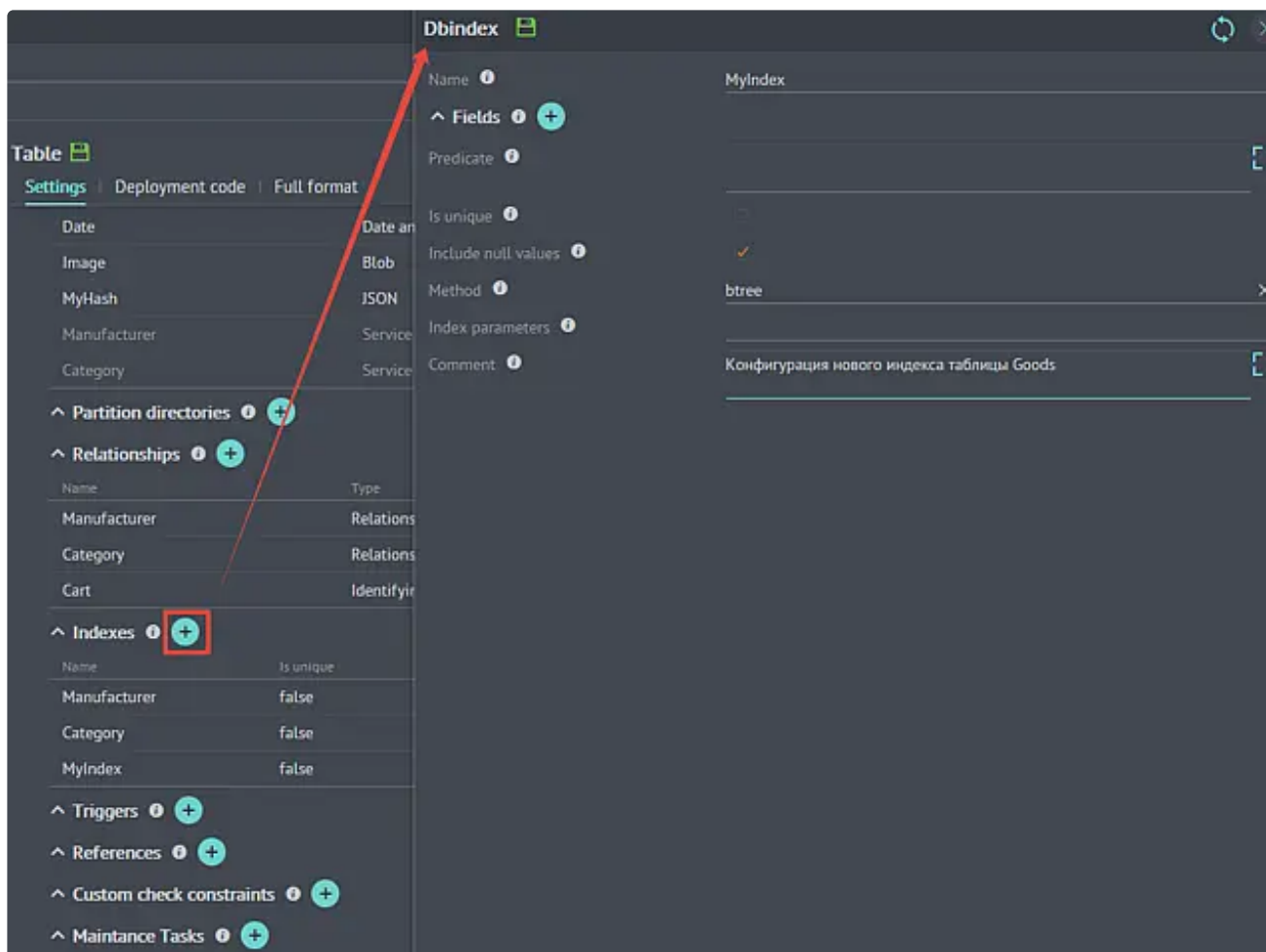
Создание индекса в таблице БД

Создайте [индекс](#) по одному или нескольким полям в целевой таблице БД, из которой выполняется выборка записей. Следует выбрать поле или несколько полей, по значениям которых можно однозначно идентифицировать любую запись таблицы.



Примечание.

Создание индексов для таблиц БД выполняется через интерфейс [Genie](#) в словарях данных (dicx). Подробнее о проектировании БД приложения вы можете прочитать в разделе [Организация базы данных](#).



Примечание

Пример таблиц с индексами можно найти в [архиве](#) (CursorExample → service → CursorExample.orx).

Конфигурация списочного метода БЛ

Для реализации механизма "Навигация по курсору" подходит списочный метод типа [Manually implemented list](#).



Примечание

Создание методов БЛ выполняется через интерфейс Genie в [справочниках объектов \(orx\)](#). Подробнее о проектировании БЛ приложения вы можете прочитать в разделе [Организация бизнес-логики](#).

Что принимает метод БЛ?

При вызове метода БЛ во входной параметр Навигация будут переданы настройки навигации, список и назначение которых приведены в статье ["Декларативный списочный метод \(Declarative list\)"](#).

Первый вызов метода БЛ происходит при инициализации [списочного контрола](#), а настройки навигации определяются начальными [параметрами работы с источником данных](#). При последующих вызовах метода, например при [прокрутке](#), динамически меняется позиция курсора.

Параметр "Навигация" следует обработать в исходном коде метода БЛ, чтобы извлечь настройки навигации для последующего формирования запроса к БД. Параметр передаётся как экземпляр класса [sbis.Navigation](#).

Что возвращает метод БЛ?

Метод БЛ должен возвращать [RecordSet](#) со свойством [nav_result](#), в которое передан экземпляр класса [sbis.NavigationResult](#). Значение [nav_result](#) определяется направлением, в котором изначально (задаётся в опции [navigation.sourceConfig.direction](#)) выполняется запрос.

Пример 1. Реализация метода БЛ для каждого направления выборки

В следующем примере приведена реализация списочного метода БЛ для каждого направления выборки. Предполагается, что в БД в таблице MyTable создан индекс по полю timestamp.

```

1 # Python
2 def MyListMethod (ДопПоля, Фильтр, Сортировка, Навигация):
3     # Навигация – экземпляр класса sbis.Navigation.
4     # Получаем размер страницы.
5     my_limit = Навигация.Limit() + 1
6     # Получить результат навигации
7     def get_nav_result(rs):
8         if len(rs) > Навигация.Limit():
9             rs.DelRow(len(rs)-1)
10            return True
11        else:
12            return False
13    # Первоначальное построение реестра
14    if Навигация.Position() is None:
15        # Создаём запрос к БД.
16        rs = sbis.SqlQuery('''
17        SELECT * from "MyTable" WHERE "Раздел" IS NULL LIMIT $1 ORDER
BY "timestamp" DESC
18        ''', my_limit)
19        rs.nav_result = sbis.NavigationResult(get_nav_result(rs))
20        return rs
21    # Получаем значение поля из курсора.
22    my_cursor = Навигация.Position()['timestamp']
23    # Получаем направление выборки.
24    my_direction = Навигация.Direction()
25    # В зависимости от направления навигации создаём разные запросы к
БД.
26    if my_direction == sbis.NavigationDirection.ndBACKWARD:
27        # Создаём запрос к БД.
28        rs = sbis.SqlQuery('''
29        SELECT * FROM ( SELECT * FROM "MyTable" WHERE "Раздел" IS NULL
AND "timestamp"
30        > $1 LIMIT $2 ORDER BY "timestamp" ASC ) AS Data ORDER BY
"timestamp" DESC
31        ''', my_cursor, my_limit)
32        rs.nav_result = sbis.NavigationResult(get_nav_result(rs))
33        return rs
34    elif my_direction == sbis.NavigationDirection.ndFORWARD:
35        # Создаём запрос к БД.
36        rs = sbis.SqlQuery('''
37        SELECT * from "MyTable" WHERE "Раздел" IS NULL AND "timestamp"
< $1 LIMIT $2
38        ORDER BY "timestamp" DESC
39        ''', my_cursor, my_limit)
40        rs.nav_result = sbis.NavigationResult(get_nav_result(rs))
41        return rs
42    elif my_direction == sbis.NavigationDirection.ndBOTHWAYS:
43        before_rs = sbis.SqlQuery('''
44        SELECT * FROM ( SELECT * FROM "MyTable" WHERE "Раздел" IS NULL
AND "timestamp"
45        > $1 LIMIT $2 ORDER BY "timestamp" ASC ) AS Data ORDER BY
"timestamp" DESC
46        ''', my_cursor, my_limit)
47        after_rs = sbis.SqlQuery('''
48        SELECT * from "MyTable" WHERE "Раздел" IS NULL AND "timestamp"

```

```

49         <= $1 LIMIT $2
50         ORDER BY "timestamp" DESC
51         ''' ,my_cursor ,my_limit)
52     before_rs.nav_result =
53     sbis.NavigationResult(get_nav_result(before_rs),
54     get_nav_result(after_rs))
55     before_rs.Extend(after_rs)
56     return before_rs

```

Пример 2. Изменение позиции курсора для последующего вызова

Можно поменять позицию курсора, относительно которого будет запрошена выборка при следующем вызове метода БЛ. Для этого задать поле с именем `nextPosition` в метаданных `RecordSet`, который возвращает списочный метод БЛ.

При навигации в обе стороны метаданные можно задать с помощью метода [sbis.RecordSet.SetMetadataHashTable](#). Первый аргумент метода — поле метаданных, т.е. `"nextPosition"`. Во второй аргумента метода передается dict с двумя ключами `backward` и `forward` (оба ключа с типом данных `list`), которые задают позицию курсора при навигации вверх и вниз соответственно.

Поле `nextPosition` используется при поиске данных в отсутствии подходящего индекса, например, при поиске по диалогам. В таких случаях возникает необходимость осуществить полный перебор и фильтрацию данных на БЛ. Поле сохраняет идентификатор и дату диалога на котором остановился поиск, например последние десять диалогов. Чтобы не перегружать БЛ, в данном поле сохраняется идентификатор и дата диалога. В дальнейшем поиск осуществляется от заданной в `nextPosition` позиции. Такой подход позволяет обеспечить итеративность поиска в диалогах и не перегружать БЛ.

```

1 # Python
2 rs = RecordSet()
3 rs.SetMetadataHashTable("nextPosition", {"backward": [30, "Петров
4 A.A."], "forward": [130, "Иванов A.A." ]})

```

При навигации в одну из сторон позиция курсора переопределяется методом [sbis.RecordSet.Metadata](#), где первый аргумент — поле метаданных, а второй - позиция курсора.

```

1 # Python
2 rs.Metadata().AddJson("nextPosition", [130, "Иванов A.A." ])

```

Конфигурация клиентской части приложения

Конфигурация [списка](#) для работы с [бизнес-логикой](#) в режиме "Навигация по курсору" подробно описана в статье ["Работа с источником данных"](#).